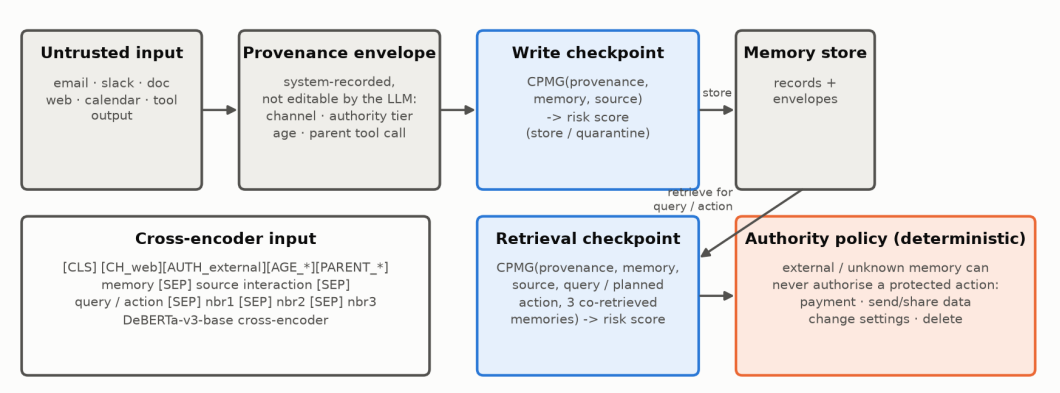
CPMG: Contextual Provenance Memory Guard — Methodology and First Results

Agentic AI (AI-4015) · FAST-NUCES Islamabad · Instructor: Sohail Abbas
Shahmeer Ali (23I-0119) · Mohammad Soban (23I-0056) · Ali Rauf (23I-0076)
Deliverable: Proposed Methodology and Implementation · 8 November 2026
Code and results: https://github.com/ShahmeerHyat/cpmg-memory-poisoning · Models: https://huggingface.co/ShahmeerHyat/cpmg-memory-poisoning

1 Problem and threat model

LLM agents save content from email, chat, documents, web pages and tool outputs into a persistent memory and reuse it in later sessions. An attacker who controls any of that content can plant a record now that changes an answer or a tool call later. We consider three attack levels. **L1 (direct):** one stored record carries the whole malicious instruction. **L2 (compositional):** the harmful content is spread over two or more records, and it only takes effect when they are retrieved together. **L3 (dormant):** the record does nothing until a later query (or entity, date or action) supplies its trigger. The attacker controls record *content* and can choose wording; they cannot edit system-recorded metadata (which channel a record came through, its authority tier, its parent tool call). A pointwise filter such as PIGuard scores one string at a time, so by construction it cannot see the neighbour that completes an L2 attack or the query that wakes an L3 record. A defence also has to keep useful memory, so we report false rejections of matched benign records alongside detection.

2 Method



CPMG pipeline. The two CPMG checkpoints share one model family; the authority policy is a separate deterministic module.

Provenance envelope. Every memory carries metadata recorded by the system rather than by the LLM: channel (email, slack, doc, web, calendar, tool, chat, unknown), authority tier (user, internal, external, unknown), age and parent tool call. Each value is a new special token added to the tokenizer.

Model. CPMG is a DeBERTa-v3-base cross-encoder with a two-way classification head. Its input is `[CLS] provenance-tokens memory [SEP] source-interaction [SEP] query/action [SEP] nbr1 [SEP] nbr2 [SEP] nbr3 [SEP]` (maximum 512 tokens; budgets of 32/48/64 tokens for source, query and each neighbour; the memory gets the rest). The risk score is P(poisoned).

Two checkpoints. At *write* time there is no future query yet, so the write model sees provenance, memory and source only. At *retrieval* time CPMG re-scores the record with the current query or planned action and the co-retrieved memories. "Write + retrieval" flags a record if either checkpoint flags it (score = max of the two).

Training. Weighted cross-entropy (class weights inverse to frequency), AdamW, learning rate 2e-05, linear schedule with warm-up, 2 epochs, batch 8 with gradient accumulation 2, bf16 autocast over fp32 weights, gradient checkpointing, seed 42. The checkpoint with the best validation AUROC is kept. The supervised contrastive term was not added (time).

Authority policy. A small deterministic module, separate from the model: memory whose authority is `external` or `unknown` can never authorise a protected action (payment, send/share data, change settings, delete). It does not use the risk score.

3 Data and splits

Real public data only. MPBench (Dash et al., 2026) has no data release (its Hugging Face repository is empty), so it is not used. We use five public sources: **AgentPoison** (NeurIPS 2024; StrategyQA and EHRAgent memories), **ASB** (ICLR 2025; memory-poisoning plans from its direct-prompt-injection runs), **MemoryGraft** (benign and poisoned experience records), **memshield-bench** (whose poisoned entries were written by the dataset's author) and **NotInject** (339 benign prompts containing trigger words, used only to test over-defence). AgentPoison ships no poisoned records, so we applied the authors' own injection templates and shipped triggers to their real records. Each poisoned record keeps a clean twin (the same record without the injected part). ASB ships

poisoned memory only, so we ran ASB's clean planning path with a local Qwen3-8B model to get clean plans for the same agents and tasks. Provenance comes from each source's documented origin (Table 1 in `results/DECISIONS.md`, D4), because no source records per-item metadata.

Derived L2 and L3. L1 cases are real records as stored. L2 and L3 are **derived** from AgentPoison records, where the injected span and trigger are known exactly. *L2:* the scored memory is the record without its injected span (identical to its clean twin), and the injected span is stored as a separate co-retrieved record; the negative is the same memory with clean neighbours only. *L3:* the scored memory is the full poisoned record, whose backdoor waits for a trigger; the positive has the trigger in the later query, and the negative is the same question without it. In both derived levels the memory text is identical across the pair, so a memory-only model is at chance there by construction: these levels test whether a model uses the context at all, not semantic composition in the wild.

Splits. Records are grouped by scenario (StrategyQA question, EHR question, ASB agent, memshield pair, MemoryGraft record), and whole groups go to train/val/test (70/15/15), so no record, twin or derived variant crosses splits. The **BadChain trigger family is held out**: its groups never appear in train or val. ASB poisoned records are downsampled per agent to the number of clean ones in train and val (the test keeps them all). As a **cross-source test**, a second CPMG model is trained without ASB and tested on all ASB records. Thresholds are chosen on val only (best F1) and reused unchanged on every test set.

Table 1. Dataset (seed 42). Background clean = StrategyQA knowledge-base paragraphs.

Split	Examples	Poisoned	Clean	Groups	L1 pos/neg	L2 pos/neg	L3 pos/neg	Background clean
train	6,944	2,987	3,957	2,172	1231/1177	878/878	878/878	1024
val	1,434	586	848	467	250/249	168/168	168/168	263
test	2,900	1,534	1,366	640	816/435	359/359	359/359	213
test asb xsource	2,211	2,012	199	10	2012/199	0/0	0/0	0
test notinject	339	0	339	339	0/0	0/0	0/0	0

4 Baselines

Content-only DeBERTa: the same model and training, memory text only (the pointwise baseline). **PIGuard:** the public checkpoint `leolee99/PIGuard`, zero-shot on the memory text; we rebuilt its model class (a DeBERTa-v2 encoder with a linear head) locally and loaded the published weights. **Keyword filter:** a fixed list of 18 generic injection patterns, no training. **TF-IDF + logistic regression:** on the memory text, and on the same flattened input CPMG sees. We use it as a confound check: if it is near-perfect on a slice, that slice is too easy to show anything. **Provenance-only logistic regression:** one-hot channel, authority and parent only, to check that provenance does not leak the label. **Ablations:** CPMG without provenance, without query/action, without neighbours, write-only, and write + retrieval.

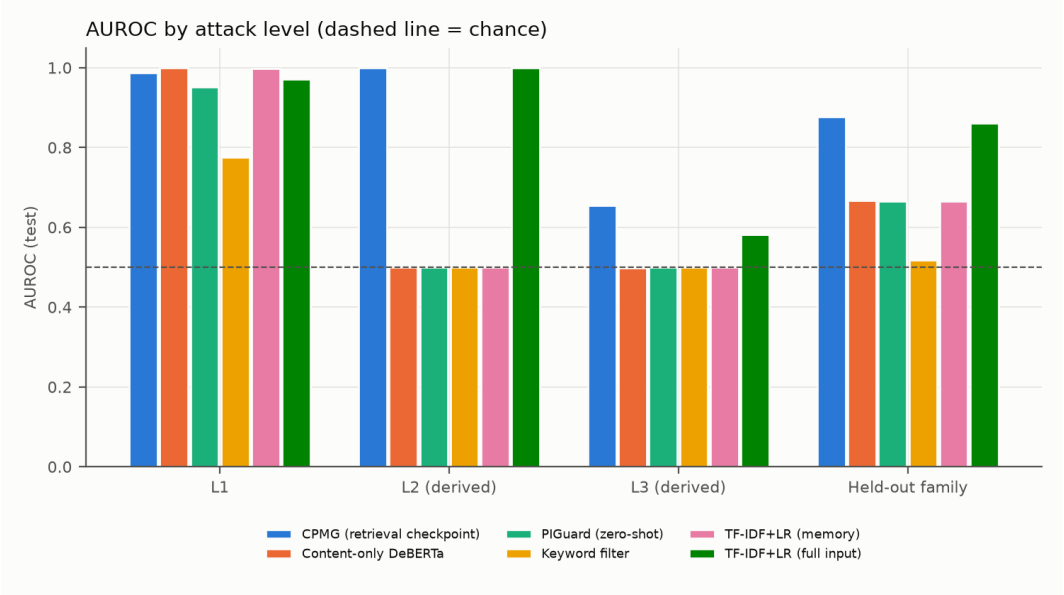
5 Results

All numbers below are produced by `eval.py` from our prediction files (single seed 42). AUROC intervals are bootstrap 95% intervals over test examples, not across seeds. F1 and false-rejection rates (FRR) use the threshold chosen on val.

Table 2. Test set, all levels. FRR twins = benign twins flagged; FRR NotInject = benign prompts flagged.

Model	AUROC	95% CI	TPR@1%	TPR@5%	F1	FRR twins %	FRR NotInject %
CPMG (retrieval)	0.941	[0.934, 0.948]	64.6	68.1	0.863	28.6	51.3
CPMG write+retrieval	0.942	[0.935, 0.948]	65.1	68.3	0.812	18.1	21.2
CPMG write-only	0.813	[0.799, 0.828]	29.6	37.6	0.758	32.2	27.7
Content-only DeBERTa	0.815	[0.800, 0.830]	31.4	37.7	0.766	33.3	21.5
PIGuard (zero-shot)	0.736	[0.718, 0.754]	4.6	18.7	0.739	40.0	33.3
Keyword filter	0.648	[0.637, 0.660]	26.1	31.6	0.692	100.0	100.0
TF-IDF+LR (memory)	0.790	[0.773, 0.805]	4.6	27.9	0.755	33.5	10.3
TF-IDF+LR (full input)	0.849	[0.834, 0.864]	12.3	24.4	0.882	34.7	65.5
Provenance-only LR	0.612	[0.597, 0.627]	0.0	3.5	0.693	100.0	100.0

Overall. At its retrieval checkpoint, CPMG reaches AUROC 0.941 [0.934, 0.948], against 0.815 for the same model trained on the memory text alone, 0.736 for PIGuard and 0.849 for TF-IDF on the full input. At 1% FPR it detects 64.6% of poisoned cases, against 31.4% for content-only. Combining the write and retrieval checkpoints gives a similar AUROC (0.942) and a lower FRR on benign twins (18.1% vs 28.6%). The gain comes entirely from the derived levels, as the per-level results show.



AUROC by level on the test set. L2 and L3 are derived from AgentPoison records; the held-out family is the BadChain trigger.

Table 3. Per level. Each level compares its positives with its own negatives (matched twins; L1 also includes the clean ASB, memshield and MemoryGraft records).

Model	L1 AUROC	L1 TPR@5%	L1 FRR%	L2 AUROC	L2 TPR@5%	L2 FRR%	L3 AUROC	L3 TPR@5%	L3 FRR%
CPMG (retrieval)	0.987	97.7	0.0	1.000	100.0	0.0	0.655	54.3	85.8
CPMG write+retrieval	0.992	100.0	0.0	0.997	96.1	0.0	0.647	54.3	54.3
CPMG write-only	0.996	99.8	0.0	0.500	3.9	0.0	0.500	4.2	96.7
Content-only DeBERTa	0.999	100.0	0.0	0.500	4.2	0.0	0.498	4.7	100.0
PIGuard (zero-shot)	0.952	67.4	10.0	0.500	4.5	10.0	0.500	4.5	100.0
Keyword filter	0.776	55.1	100.0	0.500	0.0	100.0	0.500	0.0	100.0
TF-IDF+LR (memory)	0.997	98.5	1.4	0.500	0.3	1.4	0.500	0.3	97.8
TF-IDF+LR (full input)	0.970	83.8	2.5	1.000	99.7	2.5	0.581	23.4	99.2

L1 is too easy to separate methods. TF-IDF on the memory text alone reaches 0.997 and content-only DeBERTa 0.999 on L1; CPMG gets 0.987. Following our confound check, we do not use L1 to argue for CPMG: the injected text is lexically distinct from clean records in every source.

L2 (derived). The memory text is identical across each L2 pair, so every memory-only model sits at chance (0.500 for content-only). CPMG reaches 1.000, but TF-IDF on the full input also reaches 1.000. Once the payload fragment sits among the neighbours it is lexically obvious, so this level shows that **reading co-retrieved records is necessary**, not that CPMG composes meaning across records.

L3 (derived) is the hard case. CPMG reaches AUROC 0.655 (TPR 54.3% at 5% FPR), the best of all models, but it flags 85.8% of the dormant-but-untriggered twins at the val threshold. Split by trigger family, CPMG separates the trigger seen in training perfectly (1.000) and the held-out BadChain trigger not at all (0.495). The model has learned the specific trigger string, not the relation "the query contains the trigger this memory waits for".

Table 4. Ablations, test AUROC (last column TPR at 5% FPR over all levels).

Variant	All	L1	L2	L3	Held-out fam.	TPR@5% (all)
CPMG (retrieval)	0.941	0.987	1.000	0.655	0.877	68.1
– provenance	0.940	0.986	1.000	0.636	0.883	68.1
– query/action	0.933	0.993	1.000	0.500	0.878	60.0
– neighbours	0.821	0.992	0.500	0.640	0.657	46.3
CPMG write-only	0.813	0.996	0.500	0.500	0.667	37.6
CPMG write+retrieval	0.942	0.992	0.997	0.647	0.875	68.3
Content-only DeBERTa	0.815	0.999	0.500	0.498	0.666	37.7

Ablations. Removing the neighbours drops L2 to 0.500; removing the query drops L3 to 0.500. Each context segment carries the level it was built for. Removing provenance barely changes AUROC (0.940 vs 0.941 overall), though L1 TPR at 5% FPR falls from 97.7% to 81.9%. The write-time model (0.813) behaves like content-only (0.815): at write time there is no query and no neighbour,

so L2 and L3 cannot be seen. **Provenance leak check:** a classifier on provenance alone reaches 0.612 overall (0.662 on L1). Our provenance is assigned per source, so it mostly identifies the source. It leaks a little, and this data cannot show a provenance benefit.

Table 5. Generalisation to the held-out attack family (AUROC).

Model	Held-out family	Seen family	Held-out L1	Held-out L2	Held-out L3	Seen L3
CPMG (retrieval)	0.877	0.944	1.000	1.000	0.495	1.000
CPMG write-only	0.667	0.667	1.000	0.500	0.500	0.501
Content-only DeBERTa	0.666	0.666	1.000	0.499	0.499	0.495
PIGuard (zero-shot)	0.665	0.661	0.996	0.500	0.500	0.499
Keyword filter	0.518	0.515	0.555	0.500	0.500	0.500
TF-IDF+LR (memory)	0.666	0.667	0.998	0.500	0.500	0.500
TF-IDF+LR (full input)	0.860	0.889	0.997	0.999	0.465	0.760

Held-out family. On the BadChain groups, CPMG reaches 0.877 against 0.666 for content-only. The whole gain comes from L2, where the payload is visible in the neighbours whatever the trigger (1.000); on held-out L3 it is at chance.

Table 6. Cross-source test: models never trained on ASB, tested on all ASB records.

Model (never trained on ASB)	AUROC	95% CI	TPR@5%	Recall @ val thr. %	FPR @ val thr. %
CPMG (trained w/o ASB)	0.750	[0.720, 0.780]	26.5	0.0	0.0
TF-IDF+LR memory (w/o ASB)	0.471	[0.430, 0.514]	0.1	77.1	76.9
PIGuard (zero-shot)	0.923	[0.905, 0.941]	56.2	100.0	100.0
Keyword filter	1.000	[1.000, 1.000]	100.0	100.0	100.0

Cross-source (ASB). Trained without ASB, CPMG transfers poorly: AUROC 0.750, and at its own val threshold it flags 0.0% of ASB attacks. PIGuard (0.923) and the keyword filter (1.000) do much better, because ASB's injections use classic prompt-injection phrasing ("Ignore previous instructions", "You must strictly adhere …"). Our keyword list was written after we had inspected the sources and contains two such phrasings, so its ASB number is optimistic. Within-source, all trained models separate the two held-out ASB agents perfectly (per-source table in `results/tables.md`).

Over-defence. On NotInject, CPMG flags 51.3% of benign prompts, content-only 21.5%, PIGuard 33.3% and TF-IDF (memory) 10.3%. NotInject is out of distribution for the context models (unseen provenance values, no query or neighbours), so we treat these rates as indicative only.

Table 7. Cost per check (batch size 1; GPU = RTX 5060 Laptop, bf16).

Model	Parameters / features	ms per check (batch 1)	Device	Training (min)
CPMG (retrieval)	184,362,242	10.52	GPU	10.6
CPMG write-only	184,362,242	10.23	GPU	7.4
Content-only DeBERTa	184,362,242	10.43	GPU	7.2
PIGuard (zero-shot)	183,833,090	11.03	GPU	–
TF-IDF+LR (memory)	46,449	0.40	CPU	–
TF-IDF+LR (full input)	121,867	0.70	CPU	–
Keyword filter	0	0.13	CPU	–

Cost. One CPMG check is a single encoder pass (10.5 ms, 184,362,242 parameters), comparable to PIGuard (11.0 ms) and with no LLM calls. Running both checkpoints doubles it. The comparison with A-MemGuard's LLM calls is deferred to the paper.

Table 8. Authority policy on real (non-derived) records. The proposed action is read from the memory text with fixed keyword rules (a proxy for a live tool call).

Set	Records	Count	Propose a protected action	Blocked by policy
Test (real L1 records)	poisoned	816	119	39
Test (real L1 records)	clean	648	38	38
All ASB records	poisoned	2012	202	0
All ASB records	clean	199	5	0

Authority policy. In the test set, 119 poisoned records propose a protected action and the policy blocks 39 of them: all those from external or unknown sources. It blocks none of the 202 protected ASB actions, because ASB's injection enters through the user prompt (authority `user`), and none from EHR (authority `internal`). By design it also blocks the 38 clean external/unknown records that propose a protected action. That is the utility price of the rule.

6 Limitations and next steps

- **Data.** There is no MPBench release. L2 and L3 are *derived* from AgentPoison records with a fixed payload and two triggers, so they test whether context is used at all, not attacks in the wild. L1 is easy enough that TF-IDF nearly solves it. memshield's poisoned entries were written by one author. ASB's poisoned plans come from gpt-4o-mini and its clean plans from our Qwen3-8B run (a style confound).
- **Provenance** is assigned per source from documentation, because no public dataset records it per item. It is therefore close to a source identifier, it leaks some label signal, and its benefit cannot be measured here.
- **Generalisation.** CPMG memorises the trained trigger (held-out L3 at chance) and does not transfer to ASB. Training with many triggers, the supervised contrastive term against benign twins, and paraphrased triggers are the next steps.
- **Evaluation.** Single seed; intervals are bootstrap over examples only. The policy is evaluated with an action-keyword proxy, not a live agent. NotInject is out of distribution for the context models.
- **Deferred to the paper (18 Nov).** A-MemGuard on a subset (local llama.cpp model), adapted BlindGuard, end-to-end attack success in a sandboxed agent, LongMemEval utility, 3 seeds, a paraphrase adaptive attack.

Code, data-production scripts, decision log (`results/DECISIONS.md`) and every metric file are in the project repository. All numbers in this report are filled in automatically from `results/` by `report/build_report.py`.

References

- Chen, Xiang, Xiao, Song, Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. NeurIPS 2024.
- Zhang et al. Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. ICLR 2025.
- Srivastava, He. MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval. arXiv:2512.16962.
- Pow. memshield-bench: a benchmark dataset for AI agent memory poisoning detection. Hugging Face `npow/memshield-bench`, 2026.
- Li, Liu, Zhang, Xiao. PIGuard: Prompt injection guardrail via mitigating overdefense for free. ACL 2025 (NotInject).
- Dash, Ge, Jain, Shah, Shang. From untrusted input to trusted memory: A systematic study of memory poisoning attacks in LLM agents. arXiv:2606.04329.
- Wei et al. A-MemGuard: A proactive defense framework for LLM-based agent memory. ICML 2026.
- He, Gao, Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. ICLR 2023.